Microprocessor, Microcontroller & Assembly Language
Course Code : CSE 305

Bipasha Majumder
Lecturer, CSE, DIU

What is a Assembly Language

- **Assembly language** is a low-level language that helps to communicate directly with computer hardware

- Assembly languages contain mnemonic codes that specify what the processor should do. Ex – ADD for addition, SUB for subtraction.

- The mnemonic code that was written by the programmer was converted into machine language (binary language) for execution.

- An assembler is used to convert assembly code into machine language. That machine code is stored in an executable file for the sake of execution.

Syntax of Assembly Language

- Assembly language code is generally **not case sensitive,** but we use upper' case to differentiate code from the rest of the text.

Statements : Programs consist of statements, one per line. Each statement is either an **instruction**, which the assembler translates into machine code, or an **assembler directive**, which instructs the assembler to perform some specific task, such as allocating memory space for a variable or creating a procedure. Both instructions and directives have up to four fields:

name **operation** **operand(s)** **comment**

❑ At least one blank or tab character must separate the· fields. The fields do not have to be aligned in a particular column, but they must appear in the above order.

❖ An example of an instruction is :

START: MOV CX,5 ;initialize counter

Here, the name field consists of the label START: The operation is MOV, the operands are CX and 5, and the comment is ;initialize counter.

❑ An example of an assembler directive is : MAIN PROC

MAIN is the name, and the operation field contains PROC. This particular directive creates a procedure called MAIN.

Name Field

- The name field is used for instruction labels, procedure names, and variable names.
- Names can be from 1 to 31 characters long, and may consist of letters, digits, and special characters like ?, @, _, $, %. Embedded blanks are not allowed. Names may not begin with a digit. If a period (.) is used it must be the first character.
- Some Example of legal names :

  diu_25

  diu@cse

  .Test

  Done?

- **Some Example of illegal names :**
- **TWO WORDS** **contains a blank**
- **2abc** **begins with a digit**
- **A45.28** **. not first character**
- **YOU&ME** **contains an illegal character**

Operation Field

- For an instruction, the operation field contains a symbolic operation code (opcode). The assembler translates a symbolic opcode into a machine language opcode. Opcode symbols often describe the operation's function; for example, MOV, ADD, SUB.
- In an assembler directive, the operation field contains a pseudo-operation code (pseudo-op). Pseudo-ops are not translated into machine code; rather, they simply tell the assembler to do something. For example, the PROC pseudo-op is used to create a procedure.

For an instruction, the operand field specifies the data that are to be acted on by the operation. An instruction may have zero, one, or two operands. For example,

NOP	**no operands, does nothing**
INC AX	one operand, adds 1 to the content of AX
ADD WORD1,2	two operands, adds 2 to the contents of memory words WORD1

In a two operand instruction, the first operand is the destination operand. The second operand is the source operand.

MOV Instruction

The MOV instruction is used to transfer data between registers, between a register and a memory location, or to move a number directly into a register or memory location. The syntax is,

Assembly Language	*Size*	*Operation*
MOV BL,44	8-bits	Copies a 44 decimal (2CH) into BL
MOV AX,44H	16-bits	Copies a 0044H into AX
MOV SI,0	16-bits	Copies a 0000H into SI
MOV CH,100	8-bits	Copies a 100 decimal (64H) into CH
MOV AL,'A'	8-bits	Copies an ASCII A into AL
MOV AX,'AB'	16-bits	Copies an ASCII BA* into AX
MOV CL,11001110B	8-bits	Copies a 11001110 binary into CL
MOV EBX,12340000H	32-bits	Copies a 12340000H into EBX
MOV ESI,12	32-bits	Copies a 12 decimal into ESI
MOV EAX,100Y	32-bits	Copies a 100 binary into EAX

**Note:* This is not an error. The ASCII characters are stored as a BA, so care should be exercised when using a word-sized pair of ASCII characters.

In & out Instruction

Assembly Language	*Operation*
IN AL,p8	8-bits are input to AL from I/O port p8
IN AX,p8	16-bits are input to AX from I/O port p8
IN EAX,p8	32-bits are input to EAX from I/O port p8
IN AL,DX	8-bits are input to AL from I/O port DX
IN AX,DX	16-bits are input to AX from I/O port DX
IN EAX,DX	32-bits are input to EAX from I/O port DX
OUT p8,AL	8-bits are output from AL to I/O port p8
OUT p8,AX	16-bits are output from AX to I/O port p8
OUTp8,EAX	32-bits are output from EAX to I/O port p8
OUT DX,AL	8-bits are output from AL to I/O port DX
OUT DX,AX	16-bits are output from AX to I/O port DX
OUT DX,EAX	32-bits are output from EAX to I/O port DX

Note: p8 = an 8-bit I/O port number and DX = the 16-bit port address held in DX.

Thank You